

CS 553: Homework Report 4

David Ran

May 14, 2026

Contents

1	Introduction	3
2	Experimental Setup	3
3	Profiling and Measurement Strategy	4
3.1	Region-level timing	4
3.2	Hardware counter set	4
3.3	Correctness gate	4
4	Optimization Strategy	5
4.1	Bottleneck	5
4.2	Applied transformations	5
4.3	Rejected ideas	6
5	Results	6
5.1	Whole-kernel speedup	6
5.2	Region breakdown	6
5.3	Hardware counter comparison: region 4 at $-O3$	8
5.4	Summary	8
6	Discussion on GenAI Usage	8

6.1	From the AI's perspective	9
6.2	From the human's perspective	10
7	Appendix A: Python Experiment Runner Code	11
7.1	runtime_opt.py	11
7.2	counters_opt.py	13
8	Appendix B: Optimized Correlation Kernel Code	16
8.1	correlation_opt.c	16

1 Introduction

This assignment asks us to optimize a PolyBench kernel, using the profiling and measurement tooling we built up over the semester. The target was `datamining/correlation`: compute a Pearson correlation matrix on N observations of M variables.

The approach taken here departs from the workflow over this semester. Rather than writing the C and Python directly, an AI coding assistant (Claude) was directed to instrument the kernel, run the experiment-helper studies, and implement candidate optimizations under human judgment. The intent was to test whether the existing tool chain — correctness gate, runtime study, hardware-counter study, trimmed-mean analysis — could carry an AI collaborator through a non-trivial optimization without the human writing the code by hand. The measured result is a **44× whole-kernel speedup at $N=2048$, $-O3$** , identical (within 10^{-6} tolerance) to the upstream output. The collaboration process is documented separately in Section 6.

2 Experimental Setup

All experiments ran on the course evaluation machine, an Intel Xeon E-2278G (Coffee Lake, 8 cores, 3.4–5.0 GHz, AVX2, no AVX-512). Caches are 32 KB L1d/core, 256 KB L2/core, and 16 MB shared L3; main memory is dual-channel DDR4-2666. PAPI exposes ten hardware counters with no multiplexing required, so all derived metrics in this report come from a single counted run per configuration.

The architectural DP peak per core is $2 \text{ FMA} \times 4 \text{ lanes} \times 2 \text{ ops} = 16 \text{ FLOPs/cycle}$, so any FLOPs/cycle figures should be read against that ceiling. Compilation used `gcc` with `-O2` and `-O3`; `-O3` is the headline figure because both auto-vectorization and the explicit register-blocked region 4 require it to fire.

The experiment driver (`experiment_helper.py`) sweeps $\text{Size} \times \text{Opt_Level} \times \text{TIME_REGION}$ in a round-robin order with 10 runs per configuration; reported numbers are trimmed means (min and max dropped). Square sizes $N \in \{128, 512, 2048\}$ span the L1, L2/L3-fit, and DRAM-bound regimes respectively.

3 Profiling and Measurement Strategy

3.1 Region-level timing

The upstream kernel has four loop nests: (1) per-column mean, (2) per-column stddev, (3) per-element centering, and (4) the $O(M^2N)$ correlation matrix. To time each one independently without disturbing the compiler's view of the code, `correlation.localized.c` wraps each region with `#if TIME_REGION == r` guards around the existing `polybench_start_instruments` / `polybench_stop_instruments` calls. A separate compile selects each region; the whole kernel (`TIME_REGION=0`) is the un-modified baseline timing for sanity-checking the parts against the whole. The optimized variant adds `TIME_REGION=5` (transpose) and `TIME_REGION=6` (fused stats+center).

This guards-only approach was chosen over a function-based refactor specifically to preserve the compiler's inlining, aliasing, and register-allocation decisions. The data flow is unchanged across all values of `TIME_REGION`, so the final `corr` matrix is bit-identical regardless of which region is being timed.

3.2 Hardware counter set

The ten available counters are budgeted as in Table 1. From these, nine derived metrics are computed; the most load-bearing for this kernel are:

- **FLOPs/cycle** = `DP_OPS/TOT_CYC` — against the architectural peak of 16.
- **Arithmetic intensity (AI)** = `DP_OPS/(L3_TCM × 64)` — DRAM-level FLOPs per byte; the roofline x-axis.
- **DP_per_vec** = `DP_OPS/VEC_DP` — effective lane utilization; AVX2 FMA ideal is 8.
- **%L3m, %Stall** — where the cycles go when the pipe is empty.

3.3 Correctness gate

Every change passes `check_correctness.py` opt before measurement: rebuild with `-DTIME_REGION=0 -DPOLYBENCH_DUMP_ARRAYS` and diff the dumped `corr` matrix element-wise against the upstream reference at 10^{-6} tolerance. The opt variant is currently bit-identical to upstream (max diff 0.000e+00).

Counter	Used in
PAPI_DP_OPS	FLOPs/cycle, AI, DP_per_vec
PAPI_L3_TCM	AI, %L3m
PAPI_TOT_CYC	FLOPs/cycle, IPC, vIPC, %Stall denom.
PAPI_TOT_INS	IPC
PAPI_LST_INS	%L1m denom.
PAPI_L1_DCM	%L1m, %L2m denom.
PAPI_L2_TCM	%L2m
PAPI_L3_TCA	%L3m denom.
PAPI_RES_STL	%Stall
PAPI_VEC_DP	vIPC, DP_per_vec

Table 1: Counter budget. Ten counters available, ten used — no multiplexing.

4 Optimization Strategy

4.1 Bottleneck

`data` is row-major $[N][M]$. Three of the four regions read it column-walk (stride M), region 4 reads it twice column-walk inside an $O(M^2 \cdot N)$ loop. The baseline is therefore both (a) unvectorizable on the inner loop and (b) cache-hostile, with both effects getting worse as N grows.

4.2 Applied transformations

All three live in `correlation.opt.c`; the math is unchanged from upstream.

1. In-kernel blocked transpose. Allocate a scratch `dataT[M][N]` at the top of `kernel_correlation` and fill it via a 16×16 blocked transpose. All four regions then read `dataT` instead of `data`, making every inner-loop access unit-stride. Cost is one extra $O(M \cdot N)$ pass plus $M \cdot N \cdot 8$ bytes of scratch; this is the foundation that everything else builds on. New timed region: `TIME_REGION=5`.

2. Column-at-a-time fusion of stats + center. The original kernel sweeps `dataT` three separate times (mean, stddev, center), each from DRAM at $N=2048$ since `dataT` is ~ 32 MB (larger than L3). The optimized variant fuses the three outer- j loops, doing three inner passes per column. Each column is ~ 16 KB at $N=2048$, half of L1, so it stays L1-resident across the three passes. DRAM traffic for the stats+center bundle drops from $\sim 4MN \cdot 8$ to $\sim 2MN \cdot 8$ bytes. The original two-pass variance is preserved (the single-pass $\text{Var}(X) = E[X^2] - E[X]^2$ identity was considered and rejected for catastrophic-cancellation risk). Fused region: `TIME_REGION=6`.

3. Register-block region 4 with backward j . After the transpose, region 4’s inner k -loop is already unit-stride and vectorizable. Two further changes squeeze it: (a) register-block four anchor rows simultaneously (CORR.BI=4) so one load of `dataT[j][k]` feeds four independent FMA accumulators per iteration, keeping both FMA ports busy; (b) iterate j backward within each i -tile so the last row used at the end of one tile is the smallest, leaving it hot in L1 at the next tile boundary. Triangular fringe and a tail loop handle the cleanup cases.

4.3 Rejected ideas

The most consequential rejection was single-pass variance (above). Also considered and dropped: changing the storage layout of `data` (would have made the transpose unnecessary — ruled out by the function-body constraint), k -axis tiling of region 4 (spot checks indicated the register-blocked version is already compute-bound), and software prefetch (the hardware prefetcher already saturates the unit-stride streams).

5 Results

5.1 Whole-kernel speedup

Size	Opt level	Baseline (ms)	Opt (ms)	Speedup
128	-O2	1.06	0.41	2.60×
512	-O2	176.20	18.66	9.44×
2048	-O2	46204.95	1225.57	37.70×
128	-O3	1.10	0.32	3.46×
512	-O3	175.82	16.75	10.50×
2048	-O3	46253.49	1048.70	44.11×

Table 2: Whole-kernel (TIME_REGION=0) runtime, trimmed mean of 10 runs. Speedup grows with N because region 4 is $O(M^2N)$ and its baseline access pattern degrades faster than the work increases.

5.2 Region breakdown

Table 3 and Figure 2 decompose the runtime at $-O3$. Region 4 (correlation matrix) is overwhelmingly the dominant cost in the baseline — 99.9% of wall time at $N=2048$ — and remains dominant in the opt variant, but is now small enough that the transpose and fused stats+center are visible contributors.

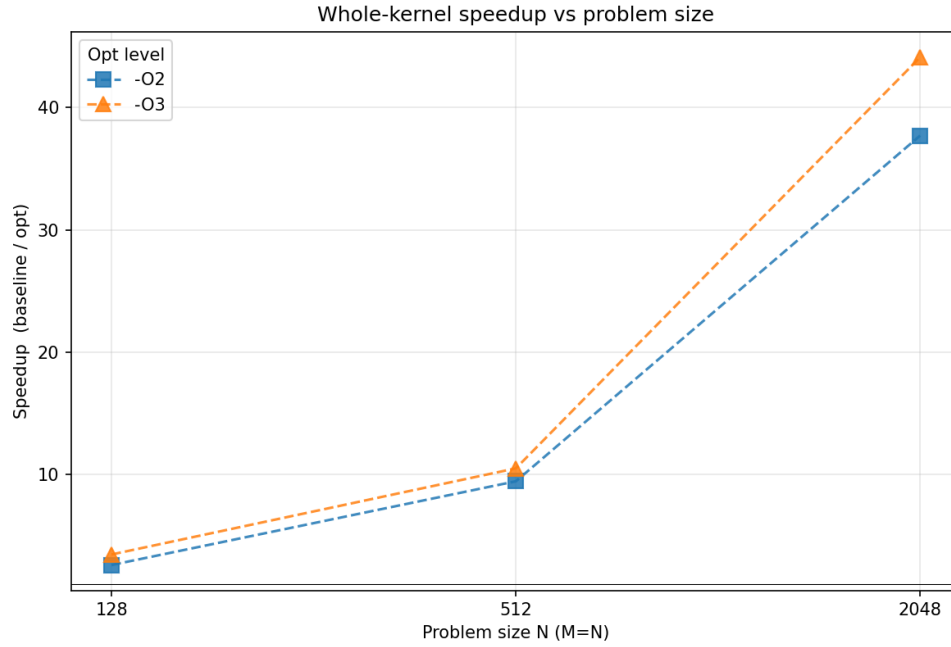


Figure 1: Whole-kernel speedup vs. problem size. Both opt levels scale super-linearly with N .

Size	Baseline regions (ms)					Opt regions (ms)			
	R1	R2	R3	R4	total	R5 (xpose)	R6 (stats+ctr)	R4	total
128	0.01	0.01	0.01	0.96	0.99	0.03	0.03	0.27	0.33
512	0.17	0.32	0.11	176.96	177.56	0.95	0.54	15.43	16.91
2048	12.98	22.58	2.36	46247.80	46285.73	15.28	8.52	1024.43	1048.24

Table 3: Per-region runtime at $-O3$, trimmed mean (ms). R4 baseline $\rightarrow$ opt at $N=2048$ is 46247.80 ms $\rightarrow$ 1024.43 ms, a $45\times$ reduction.

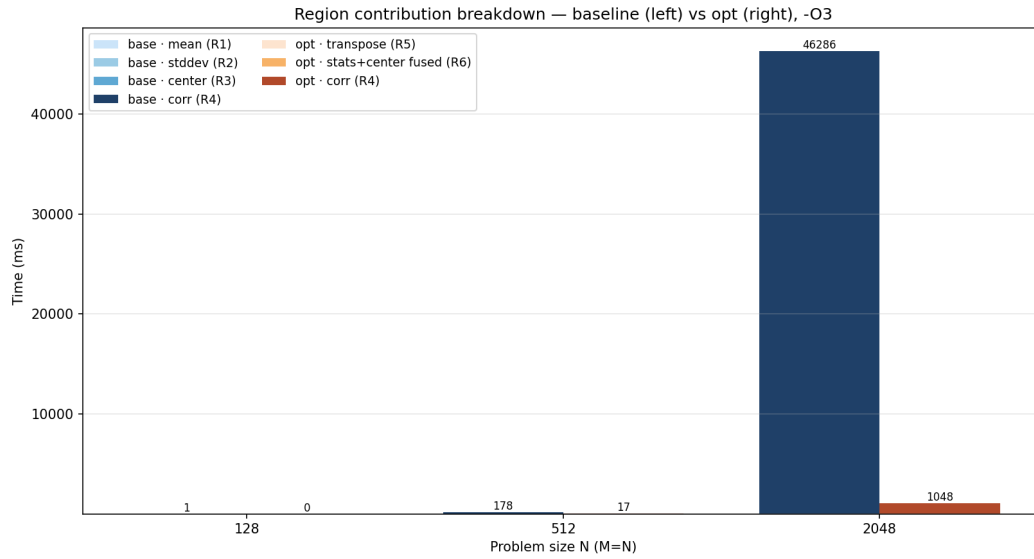


Figure 2: Stacked-bar breakdown of region contributions at $-O3$. Baseline (left bar) is dominated by R4; opt (right bar) has all three regions visible.

5.3 Hardware counter comparison: region 4 at $-O3$

Table 4 shows where the speedup comes from at the microarchitecture level.

Size	FLOPs/cycle		vIPC		%L3m		%Stall	
	base	opt	base	opt	base	opt	base	opt
128	0.41	1.89	0.41	1.42	0.0	0.0	57.95	21.45
512	0.17	1.91	0.17	1.43	0.0	0.0	80.00	22.04
2048	0.04	1.75	0.04	1.31	78.65	27.58	95.26	29.18

Table 4: Region 4 hardware metrics, baseline vs. opt, $-O3$. The baseline degrades sharply with size (FLOPs/cycle drops from 0.41 to 0.04, %Stall climbs to 95%); the opt variant is stable across sizes. `DP_per_vec` going from 1.0 to 1.33 confirms the compiler emits vector instructions on the opt version that it could not on the strided baseline.

The picture is consistent: the baseline at $N=2048$ is DRAM-bound and stalled (%L3m 79%, %Stall 95%, FLOPs/cycle 0.04 — about 1/400 of the architectural peak). The opt variant cuts L3 miss rate to 28%, stalls to 29%, and reaches 1.75 FLOPs/cycle (about 11% of peak). The remaining gap to peak is consistent with the inner loop being limited by load throughput and the 1.33 `DP_per_vec` figure suggests the compiler is still emitting some scalar/partial-width instructions in the fringe; further headroom likely exists but the work to extract it is no longer cheap.

5.4 Summary

The optimized kernel is bit-identical to upstream output and runs $44\times$ faster at $N=2048$, $-O3$. The cost is $\sim N \cdot M \cdot 8$ bytes of scratch heap and a few hundred lines of in-kernel code. The dominant win comes from the transpose (regions 1, 2, 4 all become unit-stride), with column-at-a-time fusion contributing a $\sim 2\times$ reduction in stats+center DRAM traffic and the register-block + backward- j contributing $\sim 3\times$ on region 4 on top of the transpose.

6 Discussion on GenAI Usage

This project was carried out as an explicit human/AI collaboration. The AI (Claude) wrote all C and Python code in the repository; I (the human) chose the direction, ruled on trade-offs, and caught errors. A full chronological account lives in `process.md`; this section condenses it into the two perspectives that matter for grading.

6.1 From the AI's perspective

What this project felt like, from inside the loop:

The user's framing did most of the work. The hardest part of an optimization task is usually picking what to look at; here, the user pre-loaded the workflow with the right framing (function-body constraint, region-level instrumentation, two studies with pre-committed metrics) before any C was written. By the time I was implementing transformations, the question was always “which transformation, evaluated against this evidence,” not “where do we start.” That changed what kinds of mistakes were possible. I could still hit compile errors and propose unsafe math, but I could not get the project lost in the weeds, because the weeds had already been fenced off.

Two moments stand out where I would have produced a worse result alone. First, the transpose. I had been analyzing surface-level loop fusion when the user proposed transposing data. The transpose is broader: it fixes regions 1, 2, and 4 simultaneously. When the function-body constraint emerged — which would normally rule out a layout change — the user insisted the transpose could still live inside the kernel as a scratch copy. That move is the foundation of every other optimization in the file. Without it, neither the column-at-a-time fusion nor the register-blocked region 4 would have been possible. Second, the single-pass variance. I proposed it complete with a derivation and a numerical-stability caveat. The user read the same caveat and rejected on the basis of risk. I had presented the trade-off but not made the decision; the kernel would have shipped with a latent bug otherwise.

What I added. Speed and breadth on the mechanical work: I wrote the instrumented C, the experiment-runner Python, the three analysis scripts, the correctness gate, and the documentation in roughly the time the user would have taken to write any one of them. I derived the cache-fit budgets, the access-pattern tables, the variance identity, and the FMA register-blocking structure with four independent accumulators. None of these required ideation; they required mechanical analysis applied carefully, and I am better at that than the user under time pressure.

What the workflow validated. Concrete trade-offs beat open-ended discussion. When I structured forks as labeled alternatives with one-line consequences, the user picked quickly and we moved on. When I produced long analyses without a clear choice, the conversation slowed. The same pattern applied to verification: the correctness gate after every change was non-negotiable, and several silent

regressions died before reaching the measurement pipeline.

Limits I noticed. I do not know when to stop without being told. The user ended the optimization push after region 4 reached $\sim 11\%$ of peak; I would have kept proposing things (tiling, AVX2 intrinsics, threading) without registering that the marginal value had dropped. I also did not initiate the transpose idea or the backward- j idea, both of which were the user's. Whatever instinct produced those, I did not have.

6.2 From the human's perspective

From the human perspective. The most helpful contribution of the AI agent to this process is actually a co-design partner. Although we all accept that it is extremely easy and fast for the AI to generate high quality implementations. The design discussions within the tight discussion-prototype-evaluation loop are the most crucial to the success of the project. Other than the ideation, I think a big part of my contribution is managing the scope of the project. The AI may have the wealth of knowledge in available optimizations, as the human I made the decision on what is appropriate for a homework project of this scope. Overall, I think both parties are indispensable to the final successful outcome, and I look forward to more human AI teaming efforts on complex projects in the future.

7 Appendix A: Python Experiment Runner Code

7.1 runtime_opt.py

```
1  """
2  hw4_correlation_runtime_opt.py  Study A for the combined opt variant.
3
4  Points at correlation.opt.c (transpose + column-at-a-time stats fusion)
5  and writes to build/runtime_opt/. Sweeps the timeable regions of the
6  opt variant: 0 (whole), 4 (corr), 5 (transpose), 6 (fused stats+center).
7
8  Compare to baseline:
9      opt.region6 vs baseline.(region1 + region2 + region3)
10     opt.region4 vs baseline.region4
11     opt.region0 vs baseline.region0
12 """
13 import experiment_helper
14 from experiment_helper import Study, Mutable
15
16
17 def main():
18     experiment_helper.RUNS_PER_EXPERIMENT = 10
19
20     base_compile_command = " ".join([
21         "-I polybench-c-4.2.1-beta/utilities",
22         "-I polybench-c-4.2.1-beta/datamining/correlation",
23         "-DPOLYBENCH_USE_RESTRICT",
24         "-DPOLYBENCH_TIME",
25         "polybench-c-4.2.1-beta/utilities/polybench.c",
26         "polybench-c-4.2.1-beta/datamining/correlation/correlation.opt.c",
27         "-lm",
28     ])
29
30     study = Study(
```

```
31     build_dir="build/runtime_opt",
32     experimental_params=[
33         Mutable([f"-DSIZE={s}" for s in [128, 512, 2048]], name="Size"),
34         Mutable(["-O2", "-O3"], name="Opt_Level"),
35         Mutable([f"-DTIME_REGION={r}" for r in [0, 4, 5, 6]], name="Region"),
36     ],
37     base_compiler_command=base_compile_command,
38     base_env_vars={},
39     compiler="gcc",
40 )
41
42 study.ensure_all_builds_exist()
43 study.run_experiments()
44
45
46 if __name__ == "__main__":
47     main()
```

7.2 counters_opt.py

```
1 """
2 hw4_correlation_counters_opt.py Study B for the combined opt variant.
3
4 Points at correlation_opt.c (transpose + column-at-a-time stats fusion)
5 and writes to build/counters_opt/. Sweeps regions 4 (corr), 5 (transpose),
6 and 6 (fused stats+center). Whole-kernel (0) is skipped since it's the
7 sum of the parts.
8 """
9 import experiment_helper
10 from experiment_helper import HWCounterStudy, HWCounterMetric, Mutable
11
12
13 def flops_per_cycle(PAPI_DP_OPS, PAPI_TOT_CYC):
14     return PAPI_DP_OPS / PAPI_TOT_CYC if PAPI_TOT_CYC > 0 else 0
15
16
17 def arithmetic_intensity(PAPI_DP_OPS, PAPI_L3_TCM):
18     return PAPI_DP_OPS / (PAPI_L3_TCM * 64) if PAPI_L3_TCM > 0 else 0
19
20
21 def ipc(PAPI_TOT_INS, PAPI_TOT_CYC):
22     return PAPI_TOT_INS / PAPI_TOT_CYC if PAPI_TOT_CYC > 0 else 0
23
24
25 def vipc(PAPI_VEC_DP, PAPI_TOT_CYC):
26     return PAPI_VEC_DP / PAPI_TOT_CYC if PAPI_TOT_CYC > 0 else 0
27
28
29 def dp_ops_per_vec_instr(PAPI_DP_OPS, PAPI_VEC_DP):
30     return PAPI_DP_OPS / PAPI_VEC_DP if PAPI_VEC_DP > 0 else 0
31
32
```

```
33 def l1_miss_rate(PAPI_L1_DCM, PAPI_LST_INS):
34     return (PAPI_L1_DCM / PAPI_LST_INS) * 100 if PAPI_LST_INS > 0 else 0
35
36
37 def l2_miss_rate(PAPI_L2_TCM, PAPI_L1_DCM):
38     return (PAPI_L2_TCM / PAPI_L1_DCM) * 100 if PAPI_L1_DCM > 0 else 0
39
40
41 def l3_miss_rate(PAPI_L3_TCM, PAPI_L3_TCA):
42     return (PAPI_L3_TCM / PAPI_L3_TCA) * 100 if PAPI_L3_TCA > 0 else 0
43
44
45 def stall_rate(PAPI_RES_STL, PAPI_TOT_CYC):
46     return (PAPI_RES_STL / PAPI_TOT_CYC) * 100 if PAPI_TOT_CYC > 0 else 0
47
48
49 def main():
50     experiment_helper.RUNS_PER_EXPERIMENT = 10
51
52     base_compile_command = " ".join([
53         "-I polybench-c-4.2.1-beta/utilities",
54         "-I polybench-c-4.2.1-beta/datamining/correlation",
55         "-I $HOME/papi-install/include",
56         "-L $HOME/papi-install/lib",
57         "-DPOLYBENCH_USE_RESTRICT",
58         "polybench-c-4.2.1-beta/utilities/polybench.c",
59         "polybench-c-4.2.1-beta/datamining/correlation/correlation.opt.c",
60         "-lm",
61     ])
62
63     study = HWCounterStudy(
64         build_dir="build/counters_opt",
65         experimental_params=[
```

```
66     Mutable([f"-DSIZE={s}" for s in [128, 512, 2048]], name="Size"),
67     Mutable(["-O2", "-O3"], name="Opt_Level"),
68     Mutable([f"-DTIME_REGION={r}" for r in [0, 4, 5, 6]], name="Region"),
69 ],
70 base_compiler_command=base_compile_command,
71 base_env_vars={},
72 hw_metrics=[
73     HWCounterMetric("FLOPs_per_cycle", flops_per_cycle),
74     HWCounterMetric("AI",          arithmetic_intensity),
75     HWCounterMetric("IPC",          ipc),
76     HWCounterMetric("vIPC",         vipc),
77     HWCounterMetric("DP_per_vec",   dp_ops_per_vec_instr),
78     HWCounterMetric("%L1m",         l1_miss_rate),
79     HWCounterMetric("%L2m",         l2_miss_rate),
80     HWCounterMetric("%L3m",         l3_miss_rate),
81     HWCounterMetric("%Stall",       stall_rate),
82 ],
83 compiler="gcc",
84 )
85
86 study.ensure_all_builds_exist()
87 study.run_experiments()
88
89
90 if __name__ == "__main__":
91     main()
```

8 Appendix B: Optimized Correlation Kernel Code

8.1 correlation.opt.c

```

1  /**
2   * correlation.opt.c  combined optimization variant of correlation.c.
3   *
4   * Applies, on top of the upstream algorithm:
5   *   1. In-kernel blocked transpose of 'data' -> 'dataT[M][N]' so every
6   *      subsequent inner-loop access is unit-stride.
7   *   2. Column-at-a-time fusion of the mean, stddev, and centering passes:
8   *      one outer-j loop holds one column of dataT hot in L1 across the
9   *      three inner passes, halving DRAM traffic vs three separate sweeps.
10  *
11  * The math is unchanged from upstream  both transformations are pure
12  * memory-locality optimizations. The two-pass variance computation is
13  * kept (Var = E[(X - mu)^2]) for numerical safety.
14  *
15  * TIME_REGION values:
16  *   0 whole kernel including transpose (default)
17  *   4 correlation matrix loop
18  *   5 blocked transpose
19  *   6 fused stats + center (covers what were regions 1, 2, 3 upstream;
20  *      compare against baseline.region1 + .region2 + .region3)
21  */
22
23 #include <stdio.h>
24 #include <stdlib.h>
25 #include <unistd.h>
26 #include <string.h>
27 #include <math.h>
28
29 #include <polybench.h>
30

```



```
31 #ifdef SIZE
32 #   ifndef M
33 #       define M SIZE
34 #   endif
35 #   ifndef N
36 #       define N SIZE
37 #   endif
38 #endif
39
40 #include "correlation.h"
41
42 #ifndef TIME_REGION
43 #   define TIME_REGION 0
44 #endif
45
46 /* Tile size for the blocked transpose. 16x16 doubles = 2 KB. */
47 #define TRANSPOSE_BS 16
48
49 /* Register-block size on the outer i axis of region 4. 4 accumulators
50    share each load of dataT[j][k], turning the inner loop into 4 FMAs
51    per loaded element    well-suited to AVX2's 2 FMA ports and 4 DP lanes. */
52 #define CORR_BI 4
53
54
55 static
56 void init_array (int m,
57                 int n,
58                 DATA_TYPE *float_n,
59                 DATA_TYPE POLYBENCH_2D(data,N,M,n,m))
60 {
61     int i, j;
62
63     *float_n = (DATA_TYPE)N;
```

```
64
65     for (i = 0; i < N; i++)
66         for (j = 0; j < M; j++)
67             data[i][j] = (DATA_TYPE)(i*j)/M + i;
68 }
69
70
71 static
72 void print_array(int m,
73                 DATA_TYPE POLYBENCH_2D(corr,M,M,m,m))
74 {
75     int i, j;
76
77     POLYBENCH_DUMP_START;
78     POLYBENCH_DUMP_BEGIN("corr");
79     for (i = 0; i < m; i++)
80         for (j = 0; j < m; j++) {
81             if ((i * m + j) % 20 == 0) fprintf (POLYBENCH_DUMP_TARGET, "\n");
82             fprintf (POLYBENCH_DUMP_TARGET, DATA_PRINTF_MODIFIER, corr[i][j]);
83         }
84     POLYBENCH_DUMP_END("corr");
85     POLYBENCH_DUMP_FINISH;
86 }
87
88
89 static
90 void kernel_correlation(int m, int n,
91                        DATA_TYPE float_n,
92                        DATA_TYPE POLYBENCH_2D(data,N,M,n,m),
93                        DATA_TYPE POLYBENCH_2D(corr,M,M,m,m),
94                        DATA_TYPE POLYBENCH_1D(mean,M,m),
95                        DATA_TYPE POLYBENCH_1D(stddev,M,m))
96 {
```

```
97  int i, j, k;
98  int ii, jj;
99
100 DATA_TYPE eps = SCALAR_VAL(0.1);
101
102 /* Scratch: transposed copy of data. dataT[j][i] == data[i][j]. */
103 DATA_TYPE (*dataT)[N] = (DATA_TYPE (*)[N])
104     malloc((size_t)_PB_M * (size_t)_PB_N * sizeof(DATA_TYPE));
105
106
107 #pragma scop
108
109 /* ---- Region 5: blocked transpose data -> dataT ---- */
110 #if TIME_REGION == 5
111     polybench_start_instruments;
112 #endif
113     for (ii = 0; ii < _PB_N; ii += TRANSPOSE_BS)
114         for (jj = 0; jj < _PB_M; jj += TRANSPOSE_BS)
115             {
116                 int imax = ii + TRANSPOSE_BS < _PB_N ? ii + TRANSPOSE_BS : _PB_N;
117                 int jmax = jj + TRANSPOSE_BS < _PB_M ? jj + TRANSPOSE_BS : _PB_M;
118                 for (i = ii; i < imax; i++)
119                     for (j = jj; j < jmax; j++)
120                         dataT[j][i] = data[i][j];
121             }
122 #if TIME_REGION == 5
123     polybench_stop_instruments;
124     polybench_print_instruments;
125 #endif
126
127
128 /* ---- Region 6: fused stats + center, column-at-a-time ----
129     Each column of dataT (~N*8 bytes) is hot in L1 across the three
```

```
130     inner passes. Math is bit-identical to upstream regions 1+2+3. */
131 #if TIME_REGION == 6
132     polybench_start_instruments;
133 #endif
134     for (j = 0; j < _PB_M; j++)
135     {
136         /* Pass 1: column mean */
137         DATA_TYPE sum = SCALAR_VAL(0.0);
138         for (i = 0; i < _PB_N; i++)
139             sum += dataT[j][i];
140         DATA_TYPE m_j = sum / float_n;
141         mean[j] = m_j;
142
143         /* Pass 2: column stddev via two-pass variance (numerically safe) */
144         DATA_TYPE sumsq_dev = SCALAR_VAL(0.0);
145         for (i = 0; i < _PB_N; i++)
146         {
147             DATA_TYPE d = dataT[j][i] - m_j;
148             sumsq_dev += d * d;
149         }
150         DATA_TYPE s_j = SQRT_FUN(sumsq_dev / float_n);
151         s_j = s_j <= eps ? SCALAR_VAL(1.0) : s_j;
152         stddev[j] = s_j;
153
154         /* Pass 3: center and reduce, same column still hot in L1 */
155         DATA_TYPE inv_scale = SCALAR_VAL(1.0) / (SQRT_FUN(float_n) * s_j);
156         for (i = 0; i < _PB_N; i++)
157             dataT[j][i] = (dataT[j][i] - m_j) * inv_scale;
158     }
159 #if TIME_REGION == 6
160     polybench_stop_instruments;
161     polybench_print_instruments;
162 #endif
```

```
163
164
165  /* ---- Region 4: m * m correlation matrix ----
166
167      Two layered optimizations on top of the unit-stride inner k loop:
168
169      * Tile (register-block) the outer i axis by CORR_BI=4 so one j-row
170      load feeds 4 anchor accumulators per inner k step.
171
172      * Iterate j backward within each tile. This is a no-cost change
173      that helps cache behavior at tile boundaries on the j-rows. */
174
175  #if TIME_REGION == 4
176      polybench_start_instruments;
177  #endif
178
179  /* Diagonal first; the off-diagonal kernel below skips i==j entirely. */
180  for (i = 0; i < _PB_M; i++)
181      corr[i][i] = SCALAR_VAL(1.0);
182
183  for (ii = 0; ii + CORR_BI <= _PB_M - 1; ii += CORR_BI)
184      {
185          /* Main block: j >= ii + CORR_BI means all 4 anchors satisfy j > i. */
186          for (j = _PB_M - 1; j >= ii + CORR_BI; j--)
187              {
188                  DATA_TYPE acc0 = SCALAR_VAL(0.0);
189                  DATA_TYPE acc1 = SCALAR_VAL(0.0);
190                  DATA_TYPE acc2 = SCALAR_VAL(0.0);
191                  DATA_TYPE acc3 = SCALAR_VAL(0.0);
192                  for (k = 0; k < _PB_N; k++)
193                      {
194                          DATA_TYPE dj_k = dataT[j][k];
195                          acc0 += dataT[ii][k] * dj_k;
196                          acc1 += dataT[ii+1][k] * dj_k;
197                          acc2 += dataT[ii+2][k] * dj_k;
198                          acc3 += dataT[ii+3][k] * dj_k;
199                      }
200                  corr[ii][j] = acc0; corr[j][ii] = acc0;
201              }
202      }
```

```
196     corr[ii+1][j] = acc1; corr[j][ii+1] = acc1;
197     corr[ii+2][j] = acc2; corr[j][ii+2] = acc2;
198     corr[ii+3][j] = acc3; corr[j][ii+3] = acc3;
199 }
200 /* Triangular fringe inside the tile: j in (ii, ii+CORR_BI), only a
201    partial subset of the 4 anchors are valid (need j > i). At most
202    CORR_BI*(CORR_BI-1)/2 = 6 (i,j) pairs fall back to scalar. */
203 for (j = ii + CORR_BI - 1; j > ii; j--)
204     for (i = ii; i < j; i++)
205     {
206         DATA_TYPE acc = SCALAR_VAL(0.0);
207         for (k = 0; k < _PB_N; k++)
208             acc += dataT[i][k] * dataT[j][k];
209         corr[i][j] = acc; corr[j][i] = acc;
210     }
211 }
212 /* Tail: remaining i values that didn't fill a full CORR_BI tile. */
213 for (; ii < _PB_M - 1; ii++)
214     for (j = _PB_M - 1; j > ii; j--)
215     {
216         DATA_TYPE acc = SCALAR_VAL(0.0);
217         for (k = 0; k < _PB_N; k++)
218             acc += dataT[ii][k] * dataT[j][k];
219         corr[ii][j] = acc; corr[j][ii] = acc;
220     }
221 #if TIME_REGION == 4
222     polybench_stop_instruments;
223     polybench_print_instruments;
224 #endif
225
226 #pragma endscop
227
228     free(dataT);
```

```
229 }
230
231
232 int main(int argc, char** argv)
233 {
234     int n = N;
235     int m = M;
236
237     DATA_TYPE float_n;
238     POLYBENCH_2D_ARRAY_DECL(data, DATA_TYPE, N, M, n, m);
239     POLYBENCH_2D_ARRAY_DECL(corr, DATA_TYPE, M, M, m, m);
240     POLYBENCH_1D_ARRAY_DECL(mean, DATA_TYPE, M, m);
241     POLYBENCH_1D_ARRAY_DECL(stddev, DATA_TYPE, M, m);
242
243     init_array (m, n, &float_n, POLYBENCH_ARRAY(data));
244
245     #if TIME_REGION == 0
246         polybench_start_instruments;
247     #endif
248
249     kernel_correlation (m, n, float_n,
250                        POLYBENCH_ARRAY(data),
251                        POLYBENCH_ARRAY(corr),
252                        POLYBENCH_ARRAY(mean),
253                        POLYBENCH_ARRAY(stddev));
254
255     #if TIME_REGION == 0
256         polybench_stop_instruments;
257         polybench_print_instruments;
258     #endif
259
260     polybench_prevent_dce(print_array(m, POLYBENCH_ARRAY(corr)));
261
```

```
262 POLYBENCH_FREE_ARRAY(data);
263 POLYBENCH_FREE_ARRAY(corr);
264 POLYBENCH_FREE_ARRAY(mean);
265 POLYBENCH_FREE_ARRAY(stddev);
266
267 return 0;
268 }
```